

UNIVERSIDAD CATÓLICA DEL URUGUAY

Facultad de Ingeniería y Tecnologías

Ingeniería en Informática

Informe de Proyecto

Algoritmos y Estructuras de Datos

Equipo 3

Santiago Acosta

Milagros García

Federico Martínez

Bruno Pereda

Alexis Giménez

24 de agosto de 2026

Índice

1. Introducción	2
2. Descripción general de la solución	2
2.1. Modelado de la sala de emergencias	2
2.2. Operaciones principales del sistema	4
3. Estructuras utilizadas en la implementación	5
4. Principales decisiones de diseño	6
5. Análisis de complejidad	7
6. Decisiones tomadas en el Desafío 2	9
7. Optimización realizada en el Desafío 3	10
8. Resultados de los experimentos realizados	11
9. Declaración de uso de herramientas de IA	12
10. Conclusión	13

1. Introducción

El presente proyecto tiene como objetivo aplicar los conceptos de AED a la resolución de un problema concreto. Para ello, se plantea el desarrollo de una biblioteca propia de estructuras lineales y su uso en un sistema de gestión de información, buscando de esta manera obtener una implementación funcional y también comprender cómo las decisiones sobre la representación de los datos afectan a el comportamiento del sistema.

El propósito de esta entrega es el de demostrar manejo y buen uso de las estructuras de datos dadas en el curso hasta el momento. Con el fin de emplearlo en una situación real, se plantea el escenario de la sala de emergencias. En este primer hito se pone a prueba el criterio a la hora de decidir qué estructuras utilizar y el buen manejo de estas.

A lo largo del informe se presenta el proceso del modelado del escenario y la selección de las estructuras más adecuadas para sus necesidades. Además, se describen las principales decisiones de diseño, la forma en que son integradas las estructuras desarrolladas y el comportamiento de las operaciones más relevantes del sistema. También se analiza el orden asociado a estas operaciones y la relación existente entre dicho orden y la representación que se elige.

Por último, se presenta una instancia de optimización en la cual se analiza una operación con potencial mejora, se plantea una alternativa y se comparan ambas soluciones. De esta forma, el proyecto permite observar cómo una misma necesidad se puede abordar con el uso de distintas estructuras y cómo la elección puede influir directamente en la eficiencia de la solución misma.

2. Descripción general de la solución

2.1. Modelado de la sala de emergencias

Para modelar el problema se tomó como punto de partida el flujo que atraviesa un paciente desde su llegada a la sala de emergencias hasta que termina su atención. A partir de este flujo se fueron identificando las entidades y estados que resultaban necesarios para representar la información del sistema.

La principal entidad es el paciente, identificado mediante su propio ID y con su nombre asociado, un conjunto de características, un nivel de urgencia y un estado

dentro del proceso de atención. La urgencia y el estado representan diferentes conceptos: mientras que la urgencia representa la prioridad con la que el paciente debe ser atendido, el estado muestra en qué etapa del proceso como tal se encuentra.

Los niveles de urgencia definidos son cinco:

- CRÍTICO
- URGENTE
- MODERADO
- LEVE
- NO_URGENTE

Cada nivel lleva asociado un tiempo máximo de espera que se considera tolerable. Este valor es el que permite establecer un orden de prioridad, donde un tiempo menor representa una mayor urgencia.

Por otro lado, el paciente puede atravesar cuatro estados diferentes:

REGISTRADO --> EN_ESPERA --> EN_CONSULTORIO --> ATENDIDO

Una vez que se registra, el paciente empieza en estado REGISTRADO y todavía no tiene una urgencia asignada como tal. Cuando se realiza su evaluación y se añade a la espera, se le da un nivel de urgencia y pasa a EN_ESPERA. Posteriormente, al ingresar a un consultorio pasa a EN_CONSULTORIO y una vez finalizada la atención, pasa a ATENDIDO.

La atención realizada se representa mediante Consulta, la cual conserva el identificador del paciente, su nivel de urgencia, el procedimiento realizado y la fecha en que se dio. De esta manera, la información de una atención no depende de que el paciente siga formando parte de los registros activos del sistema.

A partir de estas entidades, SalaEmergencia actúa como el componente encargado de coordinar el flujo de información. Mantiene el registro general de pacientes, la espera de atención, los consultorios disponibles y el historial de consultas realizadas.

Una decisión importante en el modelo es la diferenciación entre la información actual del sistema y la información histórica. Los pacientes pueden ser retirados de los registros activos, de la espera o de los consultorios, pero si una consulta ya fue realizada esta misma permanece en el historial. Esto permite conservar el registro de las atenciones dadas independientemente de la situación posterior del paciente.

En conjunto, el modelo busca representar no solamente quiénes son los pacientes, sino también dónde se encuentran dentro del proceso de atención de la sala y, qué información es importante que se conserve a medida que dicho proceso avanza. Esto proporciona la base sobre la cual se seleccionan las estructuras lineales que implementan cada una de estas necesidades.

2.2. Operaciones principales del sistema

A partir del modelo definido se establecieron distintas operaciones para cubrir las necesidades de la sala de emergencias. Estas operaciones no se limitan al almacenamiento o a la visualización de los datos, sino que conllevan el uso de las estructuras elegidas y permiten que el paciente avance a través de las distintas etapas posibles.

Registrar un paciente permite incorporar una nueva persona al sistema a partir de su nombre e identificador. Antes de realizar el registro se verifica que no exista otro paciente con el mismo ID. Una vez validada la condición, el paciente se añade al registro general en la posición que corresponde según su identificador y comienza con el estado REGISTRADO.

Agregar un paciente a la cola de atención permite asignarle un nivel de urgencia e incorporarlo a la ColaPrioridad. La posición que ocupará dentro de la estructura no depende en su totalidad del momento en que fue registrado, sino de su nivel de urgencia. Al realizar esta operación, el paciente pasa al estado EN_ESPERA.

Ingresar un paciente a un consultorio representa el comienzo de su atención. Para realizar esta operación se verifica que exista un consultorio disponible. El paciente es retirado de la cola de espera, incorporado a la estructura que representa los consultorios y su estado cambia a EN_CONSULTORIO. Además, la posibilidad de agregar consultorios permite ampliar la capacidad de atención cuando los recursos disponibles aumentan.

Dar de alta a un paciente completa su atención. A partir del consultorio en el que se encuentra se obtiene el paciente, se genera una nueva Consulta con el procedimiento realizado y la fecha correspondiente, y esta se incorpora al historial mediante la Pila. Finalmente, el paciente pasa al estado ATENDIDO.

Eliminar un paciente permite retirarlo de las estructuras que representan información activa. Para ello se ubica primero al paciente y luego se lo elimina de la espera, de los consultorios y del registro general. Esta operación no elimina las consultas que el paciente haya realizado anteriormente, ya que estas forman parte del historial

institucional y representan hechos que deben conservarse.

Además de estas operaciones vinculadas al flujo de atención, el sistema incorpora distintas consultas de información. Buscar paciente permite localizar un paciente a partir de su identificador, mientras que listar pacientes, listar consultas y mostrar pacientes en consultorios permiten consultar el contenido de las distintas áreas del sistema. El método `toString` de `SalaEmergencia` ofrece además una visión resumida del estado general, mostrando la cantidad de pacientes registrados, en espera, en consultorios y las consultas realizadas.

3. Estructuras utilizadas en la implementación

Una vez definido el modelo de la sala de emergencias, se seleccionaron las estructuras que mejor se adaptaban al comportamiento esperado de cada conjunto de información. La elección no se hizo solamente considerando qué estructura podía almacenar los datos, sino más bien qué operaciones serían más frecuentes y qué comportamiento debía representar cada parte del sistema.

Para el registro general de pacientes se utiliza una `ListaArray` de pacientes. En la versión final, esta lista es ordenada por el identificador del paciente. Esto permite acceder directamente a cualquier posición y, al mantener los elementos ordenados, utilizar búsqueda binaria para localizar pacientes. El acceso por índice tiene un costo $O(1)$, mientras que la búsqueda binaria permite realizar las búsquedas por identificador en $O(\log n)$. Esta estructura también se utiliza para representar los consultorios, aunque en este caso la elección responde a una necesidad diferente: los consultorios constituyen un recurso físico limitado y numerado, por lo que el acceso mediante una posición representa naturalmente al consultorio correspondiente.

La espera de atención se implementa mediante una `ColaPrioridad` de pacientes. Una cola convencional seguiría un comportamiento FIFO, pero esto no representaría adecuadamente la situación planteada, ya que un paciente con mayor urgencia debe ser atendido antes que otro que haya llegado anteriormente. La `ColaPrioridad` mantiene los pacientes ordenados según el nivel de urgencia, utilizando como criterio el tiempo máximo de espera tolerable asociado a cada nivel. La inserción requiere recorrer la estructura hasta encontrar la posición correspondiente y tiene un costo $O(n)$, mientras que consultar o retirar al paciente que se encuentra al frente tiene un costo $O(1)$.

El historial de consultas se representa mediante una `Pila` de consultas. Esta estruc-

tura responde al comportamiento LIFO, de modo que la última consulta registrada queda disponible primero. Esto permite consultar las atenciones más recientes de manera natural y realizar las operaciones principales de la pila en $O(1)$.

Por último, cada paciente posee una ListaEnlazada de características. En este caso no se requiere acceso directo por posición ni una búsqueda optimizada, sino una estructura sencilla que permita agregar y eliminar características. La lista enlazada resulta suficiente para representar esta colección y, además, forma parte de la biblioteca de estructuras desarrollada durante el primer desafío.

De esta manera, las estructuras utilizadas no cumplen todas la misma función. La ListaArray se aprovecha principalmente por su acceso por índice y, en el caso de los pacientes registrados, por la posibilidad de mantener los elementos ordenados, la ColaPrioridad representa la necesidad de establecer un orden de atención, la Pila representa el historial de consultas, y la ListaEnlazada permite almacenar las características asociadas a cada paciente. La solución utiliza así distintas representaciones para necesidades que, aunque pertenecen al mismo sistema, presentan comportamientos diferentes.

4. Principales decisiones de diseño

Una de las principales decisiones fue representar la espera mediante una cola de prioridad en lugar de una cola FIFO convencional. En una sala de emergencias, el orden de llegada por sí solo no determina necesariamente el orden de atención. Por este motivo, cada paciente recibe un nivel de urgencia al incorporarse a la espera y la cola utiliza ese nivel para establecer su posición. Los pacientes de mayor urgencia quedan delante de aquellos con menor urgencia y, cuando dos pacientes tienen el mismo nivel, se conserva el orden en el que fueron incorporados.

También se decidió separar el nivel de urgencia del estado del paciente. La urgencia describe la prioridad con la que debe ser atendido, mientras que el estado indica en qué etapa del proceso se encuentra. Un paciente puede, por ejemplo, ser URGENTE y encontrarse EN_ESPERA, o continuar siendo URGENTE mientras se encuentra EN_CONSULTORIO. Mantener ambos conceptos separados evita mezclar dos características diferentes del dominio y permite conocer la prioridad como la situación actual de cada paciente.

La representación de los consultorios responde a otra característica particular del problema. Se utiliza una ListaArray con una capacidad inicial de cinco posiciones,

donde cada posición representa un consultorio. Esto permite asociar directamente el número de consultorio con el índice de la estructura y facilita verificar si existe espacio disponible antes de ingresar un nuevo paciente. Además, se incorporó la posibilidad de ampliar la capacidad cuando sea necesario, representando de forma sencilla una eventual ampliación de los recursos de la sala.

Otra decisión importante fue mantener el historial de consultas separado del registro activo de pacientes. Cuando un paciente termina su atención, se genera una Consulta que contiene su identificador, nivel de urgencia, procedimiento realizado y fecha, y esta se incorpora al historial. Si posteriormente el paciente es eliminado del sistema, su consulta no se elimina. Esto responde a una situación propia del dominio: una atención ya realizada constituye un hecho que debe permanecer registrado, independientemente de que el paciente siga siendo parte o no de los registros activos del sistema.

Respecto a la identificación de los pacientes, se decidió utilizar el ID como criterio único. Los métodos equals y hashCode de Paciente se basan en este identificador, permitiendo que dos instancias que representen al mismo paciente sean consideradas iguales aunque sean objetos diferentes en memoria. Esto resulta especialmente importante al buscar o eliminar pacientes de las distintas estructuras utilizadas por la sala.

Finalmente, durante el desarrollo se tomó la decisión de cambiar la representación del registro general de pacientes. Inicialmente se utilizaba una ListaEnlazada, que permitía incorporar nuevos pacientes al final en $O(1)$, pero obligaba a realizar búsquedas lineales en $O(n)$. Al revisar el uso de la operación de búsqueda, se identificó que esta era utilizada desde distintas operaciones del sistema, como registrar, agregar un paciente a la cola y eliminarlo. Por este motivo, se optó por una ListaArray ordenada por ID, sacrificando eficiencia en la inserción para permitir realizar búsquedas mediante búsqueda binaria.

5. Análisis de complejidad

El análisis de complejidad permite relacionar las operaciones del sistema con las características de las estructuras elegidas. En particular, resulta importante distinguir entre el costo de una operación dentro de una estructura y el costo total de una operación de la sala, ya que una operación del sistema puede involucrar varias estructuras.

La operación `buscarPaciente` constituye uno de los casos más relevantes. En la implementación final, los pacientes se encuentran almacenados en una `ListaArray` ordenada por identificador. Como esta estructura permite acceder a cualquier posición en $O(1)$, se puede aplicar búsqueda binaria sin que el acceso a los elementos se convierta en un costo adicional. La búsqueda descarta aproximadamente la mitad de los elementos en cada iteración, por lo que su complejidad es $O(\log n)$, donde n representa la cantidad de pacientes registrados.

`RegistrarPaciente` también utiliza esta búsqueda para verificar que el identificador no esté repetido y determinar la posición en la que debe incorporarse el nuevo paciente. La búsqueda del punto de inserción tiene un costo $O(\log n)$, pero la inserción en una `ListaArray` puede requerir desplazar los elementos posteriores para mantener el orden. En el peor caso, este desplazamiento cuesta $O(n)$, por lo que la complejidad total de registrar un paciente es $O(n)$.

La operación `agregarPacienteACola` requiere primero localizar al paciente mediante `buscarPaciente`, con un costo $O(\log n)$, y posteriormente incorporarlo a la `ColaPrioridad`. La inserción en esta estructura requiere recorrer los pacientes hasta encontrar la posición determinada por la prioridad, por lo que tiene un costo $O(n)$. En consecuencia, el costo dominante de la operación completa es $O(n)$.

Para `ingresarPaciente`, retirar al paciente de la `ColaPrioridad` implica una búsqueda dentro de la lista enlazada utilizada por la cola, por lo que esta operación tiene un costo $O(n)$. La incorporación del paciente a los consultorios mediante `ListaArray` es $O(1)$ si existe espacio disponible. Por lo tanto, el costo total está dominado por la eliminación de la cola y es $O(n)$.

`DarDeAlta` utiliza el acceso por posición de la `ListaArray` de consultorios y la incorporación de una nueva consulta a la Pila. Ambas operaciones principales tienen costo $O(1)$. Por lo tanto, la operación de alta tiene una complejidad temporal $O(1)$, sin considerar operaciones adicionales de salida o formato.

`EliminarPaciente` combina varias operaciones. Primero realiza una búsqueda binaria en el registro general, con costo $O(\log n)$. Luego intenta eliminar al paciente de la `ColaPrioridad`, cuyo costo es $O(n)$, y de los consultorios mediante una búsqueda y eliminación en la `ListaArray`, también $O(n)$. Finalmente, eliminarlo del registro general requiere localizarlo nuevamente dentro del array y desplazar los elementos posteriores, lo que puede costar $O(n)$. Por lo tanto, el costo total de eliminar un paciente es $O(n)$.

Las operaciones de consulta también dependen de la representación utilizada. El acceso a un paciente concreto por su identificador se beneficia de la búsqueda binaria, mientras que listar una estructura completa requiere recorrer sus elementos y tiene un costo $O(n)$. El método `toString` de `ListaEnlazada` fue implementado recorriendo directamente sus nodos, por lo que su costo es lineal respecto de la cantidad de elementos, evitando realizar accesos repetidos por posición.

En términos generales, la solución presenta un comportamiento en el que las estructuras fueron seleccionadas buscando que las operaciones más importantes tengan un costo acorde a su frecuencia y función. La `ListaArray` permite optimizar las búsquedas sobre pacientes registrados, la `ColaPrioridad` concentra el costo en la organización de la espera y la `Pila` permite registrar nuevas consultas en tiempo constante. Esta relación entre representación y complejidad es precisamente la que permite posteriormente identificar oportunidades de optimización dentro del sistema.

6. Decisiones tomadas en el Desafío 2

Durante el Desafío 2, el principal objetivo fue transformar las necesidades planteadas para una sala de emergencias en un modelo que pudiera resolverse utilizando exclusivamente las estructuras desarrolladas en el Desafío 1. Esto implicó decidir qué información debía conservarse, qué operaciones debía ofrecer el sistema y qué estructura representaba mejor cada necesidad.

Una de las primeras decisiones fue separar el registro general de pacientes de la estructura utilizada para gestionar la espera. Un paciente puede estar registrado en el hospital sin encontrarse necesariamente esperando atención. Por este motivo, el registro y la cola representan conceptos diferentes y se mantienen en estructuras independientes.

También se incorporó el concepto de estado del paciente. Los estados `REGISTRADO`, `EN_ESPERA`, `EN_CONSULTORIO` y `ATENDIDO` permiten conocer en qué etapa del proceso se encuentra cada persona sin tener que inferirlo únicamente observando las estructuras. Esto resulta especialmente útil porque un paciente puede pasar de una estructura a otra durante su atención, mientras que el registro general permanece como referencia del paciente.

La urgencia se modeló independientemente del estado. Los cinco niveles definidos determinan la prioridad dentro de la espera. Cada nivel posee asociado un tiempo máximo de espera tolerable, utilizado como criterio por la `ColaPrioridad`. De esta

manera, la estructura de datos no solo almacena los pacientes, sino que representa directamente una de las reglas fundamentales del funcionamiento de la sala.

Otra decisión fue incorporar los consultorios como una estructura independiente. Aunque técnicamente podría haberse retirado al paciente de la cola y registrar directamente su consulta al finalizar la atención, representar los consultorios permite modelar una etapa intermedia real del proceso. Además, permite controlar la capacidad disponible y asociar cada paciente con un número de consultorio concreto.

Finalmente, se decidió conservar el historial de consultas incluso cuando un paciente es eliminado del registro activo. La eliminación representa que el paciente deja de formar parte de la gestión actual de la sala, pero no significa que sus atenciones anteriores hayan dejado de existir. Mantener estas consultas permite conservar información relevante sobre los procedimientos realizados y hace que el comportamiento del sistema sea más cercano al de un registro hospitalario real.

Estas decisiones permitieron que el sistema no se limitara a almacenar pacientes, sino que representara el proceso completo desde su registro hasta la finalización de la atención, utilizando las estructuras lineales como parte del propio modelo.

7. Optimización realizada en el Desafío 3

Durante el Desafío 3 se analizó el costo de las operaciones implementadas con el objetivo de identificar alguna parte del sistema que pudiera convertirse en un cuello de botella. El principal problema encontrado fue la búsqueda de pacientes.

En la primera representación, los pacientes registrados se almacenaban en una `ListaEnlazada`. Para encontrar un paciente era necesario recorrer los nodos desde el comienzo hasta localizar el identificador buscado. Por lo tanto, `buscarPaciente` tenía una complejidad $O(n)$, siendo n la cantidad de pacientes registrados. Este costo no se producía únicamente cuando el usuario solicitaba una búsqueda: la operación también era utilizada internamente por otras funcionalidades, como registrar un paciente, agregarlo a la cola y eliminarlo.

A partir de este análisis se consideró que mejorar la búsqueda podía tener un impacto significativo sobre el sistema. La alternativa elegida fue cambiar la representación del registro de pacientes a una `ListaArray` ordenada por identificador.

Esta modificación permitió aprovechar dos características de la estructura. Por un lado, `ListaArray` proporciona acceso directo a una posición mediante un índice, con

costo $O(1)$. Por otro, al mantener los pacientes ordenados por ID, es posible aplicar búsqueda binaria. En lugar de recorrer los pacientes uno por uno, la búsqueda compara el identificador con el elemento central y descarta aproximadamente la mitad de los elementos restantes en cada iteración. De esta manera, `buscarPaciente` pasó de $O(n)$ a $O(\log n)$.

El cambio, sin embargo, introdujo un costo adicional en el registro. En la `ListaEnlazada` original, agregar un paciente al final tenía un costo $O(1)$, ya que la estructura mantiene una referencia a su último nodo. Con la nueva representación, los pacientes deben mantenerse ordenados, por lo que registrar un paciente puede requerir desplazar elementos para generar espacio en la posición correspondiente. Esta inserción tiene un costo $O(n)$ en el peor caso.

La optimización, por lo tanto, no consiste en mejorar todas las operaciones simultáneamente, sino en realizar un intercambio entre costos. Se acepta una inserción más costosa para obtener búsquedas considerablemente más eficientes. La decisión se fundamenta en el uso que tiene la búsqueda dentro del sistema: registrar un paciente ocurre una vez por cada paciente incorporado, mientras que `buscarPaciente` puede ser ejecutado repetidamente tanto directamente como desde otras operaciones.

El resultado es una representación más adecuada para el comportamiento esperado del sistema. La tabla conceptual de la optimización puede resumirse de la siguiente manera: la búsqueda de pacientes pasó de $O(n)$ a $O(\log n)$, mientras que el registro pasó de $O(1)$ en la inserción a $O(n)$ en el peor caso. La mejora se obtiene priorizando la operación que tiene mayor presencia dentro del funcionamiento general de la sala.

Esta modificación también muestra uno de los principios centrales del proyecto: una estructura no es mejor de forma absoluta que otra. Su conveniencia depende de las operaciones que el sistema necesita realizar con mayor frecuencia. En este caso, la `ListaArray` ordenada resulta preferible porque permite explotar la búsqueda binaria y reducir el costo de una operación que se utiliza en múltiples partes de la solución.

8. Resultados de los experimentos realizados

Tras calcular los órdenes de tiempo de ejecución de todas las operaciones que se verían afectadas por el cambio de representación, se compararon ambas alternativas para el registro general de pacientes y se evaluó qué implicaba cada una para el funcionamiento del sistema.

La conclusión a la que se llegó es que, si bien el registro de un paciente pasa a tener un costo $O(n)$ — tanto por el desplazamiento de los elementos posteriores que exige mantener el orden como por el eventual redimensionamiento del arreglo cuando se agota su capacidad —, ese costo se paga una única vez por cada paciente incorporado. La búsqueda por identificador, en cambio, pasa a resolverse en $O(\log n)$ en lugar de $O(n)$, y es una operación que el sistema ejecuta con una frecuencia considerablemente mayor: no solamente cuando el usuario la solicita de forma explícita, sino también de manera interna desde registrarPaciente, agregarPacienteACola y eliminarPaciente. La mejora alcanza así a todos los métodos que la emplean y su efecto se vuelve más notorio a medida que el registro acumula pacientes.

A esto se suma una ventaja en el uso de memoria. En la ListaEnlazada, cada elemento almacenado requiere un nodo que además del paciente conserva una referencia al siguiente, mientras que la ListaArray mantiene las referencias de forma contigua dentro del arreglo. La representación optimizada utiliza, por lo tanto, menos espacio por instancia de objeto almacenado.

En conjunto, la comparación respalda la decisión tomada: el costo adicional se concentra en una operación puntual y acotada, mientras que el beneficio se distribuye sobre la operación que la solución utiliza con mayor frecuencia y desde una mayor cantidad de puntos.

9. Declaración de uso de herramientas de IA

Durante el desarrollo del proyecto se utilizaron herramientas de IA generativa como apoyo en tres aspectos puntuales de la implementación. En todos los casos el código incorporado fue revisado, comprendido y probado por el equipo, mientras que las decisiones de modelado y de selección de estructuras fueron tomadas íntegramente por los integrantes.

El primer caso corresponde a la definición del criterio de orden de la cola de prioridad. La decisión de ordenar a los pacientes según el tiempo máximo de espera tolerable asociado a su nivel de urgencia, de modo que un tiempo menor represente una mayor prioridad, fue tomada por el equipo como parte del modelado del problema. Sin embargo, no conocíamos la forma de expresar ese criterio en Java para poder entregárselo a la ColaPrioridad, por lo que se consultó cómo construir un comparador a partir de un atributo numérico de una clase. De esa consulta surgió la constante `POR_URGENCIA` de `SalaEmergencia`, que obtiene de cada pacien-

te el tiempo máximo de espera de su nivel de urgencia y compara a los pacientes por ese valor en orden ascendente. La ColaPrioridad recibe ese comparador en su constructor y lo utiliza para determinar la posición que ocupa cada paciente al ser incorporado.

El segundo caso corresponde a la implementación de los métodos toString. Se utilizó la herramienta como apoyo para su escritura, particularmente en el uso de StringBuilder y en el formato de la salida, mientras que la información que debía mostrarse en cada caso fue definida por el equipo. Cabe mencionar que la primera versión del toString de ListaEnlazada recorría la estructura mediante accesos repetidos por posición, lo que producía un costo cuadrático respecto de la cantidad de elementos. El equipo identificó ese problema y reescribió el método recorriendo directamente los nodos de la lista, obteniendo así un costo lineal.

No se utilizaron herramientas de IA para el modelado de la sala de emergencias, la selección de las estructuras de datos, la implementación de los TDA ni el análisis de complejidad presentado en este informe.

10. Conclusión

El desarrollo de este proyecto permitió aplicar las estructuras lineales implementadas durante el curso a un escenario concreto y observar de qué manera la representación elegida condiciona el comportamiento del sistema. El trabajo no consistió únicamente en almacenar información, sino en decidir qué estructura representaba mejor cada parte del proceso de atención, considerando las operaciones que debían realizarse con mayor frecuencia sobre cada conjunto de datos.

A lo largo del proceso quedó en evidencia que una misma necesidad puede resolverse de distintas maneras y que ninguna estructura resulta mejor que otra de forma absoluta. La ListaArray ordenada por identificador permitió reducir el costo de la búsqueda de pacientes, aunque a cambio de una inserción más costosa, y esa decisión solo se justifica por el uso que la búsqueda tiene dentro del sistema. La cola de prioridad, por su parte, muestra cómo una estructura puede representar directamente una regla del dominio, en este caso el orden en que los pacientes deben ser atendidos.

El análisis de complejidad resultó ser una herramienta útil no solo para describir la solución alcanzada, sino para identificar dónde convenía modificarla. Reconocer que la búsqueda de pacientes se utilizaba desde varias operaciones del sistema fue lo que

permitió detectar el principal punto de mejora y fundamentar el cambio realizado. En ese sentido, el proyecto permitió comprender que la elección de una estructura forma parte del diseño de la solución y no es una decisión posterior a él.

Más allá de lo técnico, la experiencia resultó muy enriquecedora para el equipo. Para algunos integrantes significó aplicar por primera vez conocimientos adquiridos durante este semestre, mientras que para otros implicó volver a trabajar con conceptos vistos en semestres anteriores, esta vez en el contexto de un caso cercano a una situación real. Ese contraste de puntos de partida dentro del grupo hizo que la discusión sobre qué estructura utilizar en cada caso fuera una parte importante del trabajo y no solamente un paso previo a la implementación.